



MICHIGAN ENGINEERING
UNIVERSITY OF MICHIGAN

LLM Inference

NanoFlow ('25) and MOONCAKE ('25)

Alan, Evan, Madeleine, Frank
February 19, 2026



Agenda:

- Background/Introduction
- NanoFlow ('25)
- MOONCAKE ('25)
- Conclusion and Thoughts
- Q&A Discussion



Prefill and Decode

Prefill

- Parallel attention over tokens
- Compute-heavy (GEMMs)

Decode

- Sequential with attention over previous tokens
- Memory-heavy, latency critical



Bounding Operations (Roofline Model)

- *Performance* = $\min(\text{Compute Bound}, \text{Memory-Bound})$
- Arithmetic Intensity (AI) = (FLOPs/Bytes Moved)
- Prefill sits near the compute roof
- Decode near memory roof



KV Cache

- Stores Key Value vector pairs
- Avoid recomputing attention for prefix



MICHIGAN ENGINEERING
UNIVERSITY OF MICHIGAN

NanoFlow (2025)

**Towards Optimal Large Language Model Serving
Throughput**

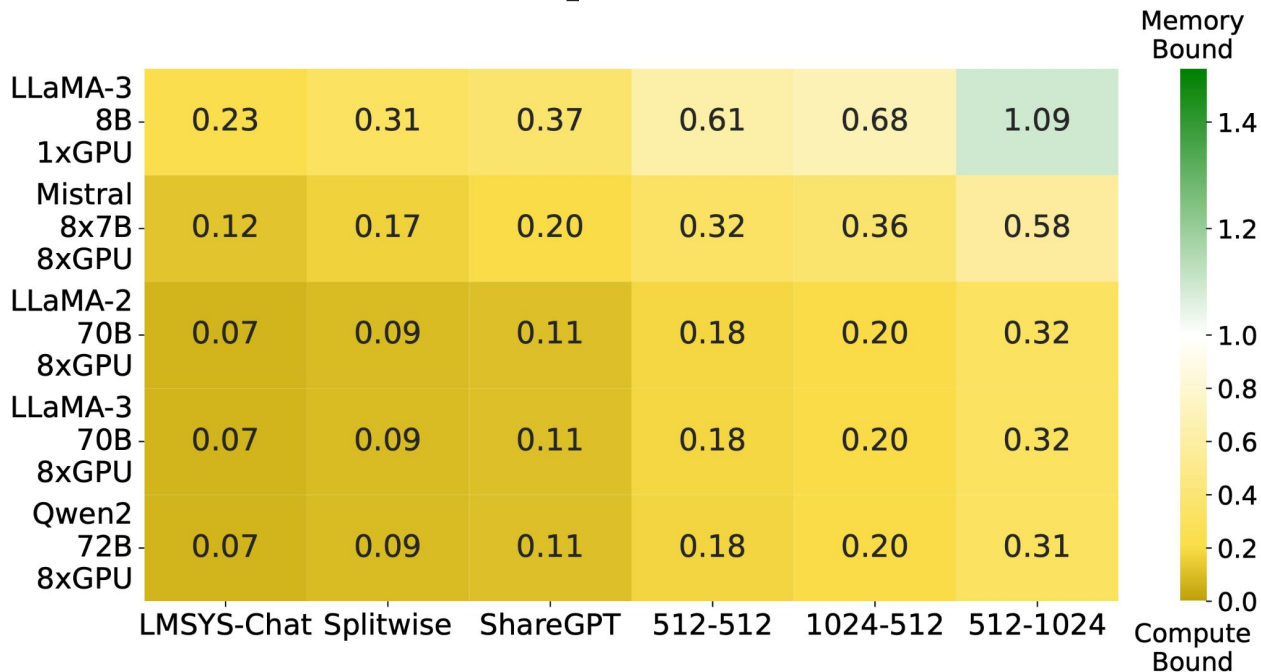


Compute-bound vs Memory-bound

- Nanoflow $T = \max(T_{compute}, T_{memory}, T_{network})$
- Compute ***Dominates over end to end pipeline***
- Batching changes the game
- Grouped Query Attention (GQA)



NanoFlow - Compute-Bound





NanoFlow - Compute-Bound





Nanobatches

- Problem
 - Decode iterations execute sequentially
 - GPU compute underutilized
- Idea
 - Split each iteration into smaller independent chunks
 - Execute chunks from different iterations in parallel



Nanobatches

- Fine-grained slice of a model iteration
- Overlap compute-heavy and memory-heavy work
- Increase tensor core utilization



Nano-Operators

- Transformer layers contain heterogeneous operations
 - Decompose layers into smaller operators
 - Treat them as schedulable units
 - Nano-operator = a fine-grained sub-operation within a transformer layer
 - Interleave nano-operators across nanobatches, and overlap compute-bound + memory-bound work
- Examples
 - QKV projection
 - Attention score computation



NanoFlow

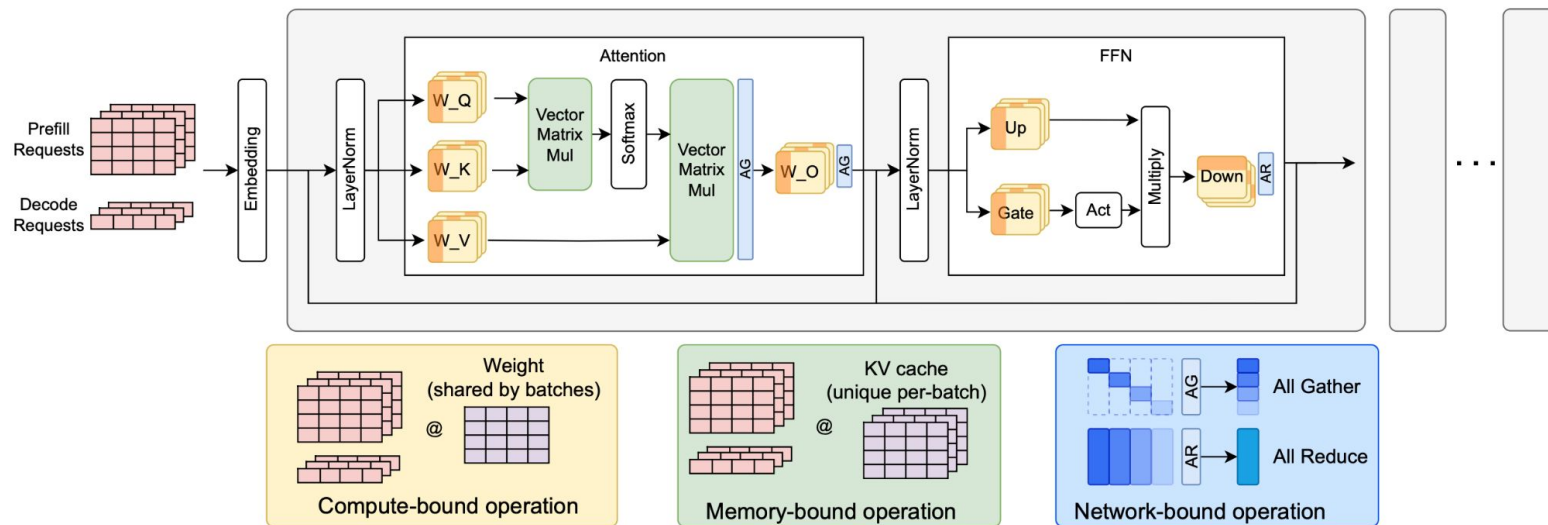
Nano-flow design

Nano batches

Auto-search algorithm

Experiments and Evaluation

NanoFlow





Key Factors of Serving Throughput

- Hardware specification
- Model configuration
- User query statistics
- Batch size



NanoFlow Design Overview

- **Automated Pipeline Search**
 - Kernel Profiling and Interference Modeling
 - Auto-search Stage I: Pipeline Structure Search
 - Auto-search Stage II: Refining the Pipeline
- **NanoFlow Runtime**
 - Request Scheduling
 - KV-cache Management



Automated Pipeline Search Motivation

- Different GPU's and model's have different preferences for:
 - Number of Nano-Batches
 - Size of Nano-Batches
 - Ordering of Nano-Operations
- **Idea:** automatically search for the best strategy



Automated Pipeline Search Phases

- **Phase I:**
 - Drafts schedule that minimizes pipeline bubbles
 - Assumes interference-free runtimes
 - Determines nano-batch splits, ordering, and overlap decisions
- **Phase II:**
 - Adds resource allocation to schedule, accounting for performance degradation
 - Refines execution time to minimize pipeline latency



Kernel Profiling and Interference Modeling

- Determine the maximum dense batch size
 - Limited by GPU memory (weights + KV cache)
 - Sets upper bound for profiling batch size
- Profiling interference-free kernels
 - Run compute, memory, and network kernels in isolation
 - For each (kernel, batch size), record best runtime



Interference Modeling (cont.)

- Profiling kernels with interference
 - Run kernels pairwise with compute
 - Measure slowdown to estimate resource share R
 - Build interference mapping: $R \rightarrow P$

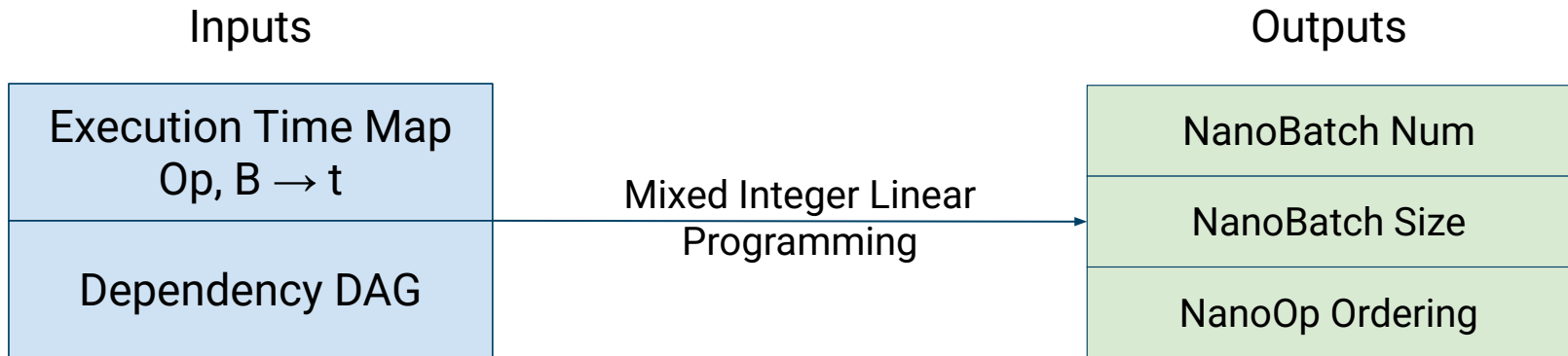
Table 3: Performance P of GEMV and network kernels with different resource utilization R .

Operations	Resource Utilization (R)						
	0	0.1	0.2	...	0.8	0.9	1
GEMM (by definition)	0	0.1	0.2	...	0.8	0.9	1
GEMV	0	0.2	0.3	...	0.85	0.95	1
Network	0	0.3	0.5	...	0.9	1	1



Stage I: Pipeline Structure Search

- Goal: **remove pipeline bubbles**





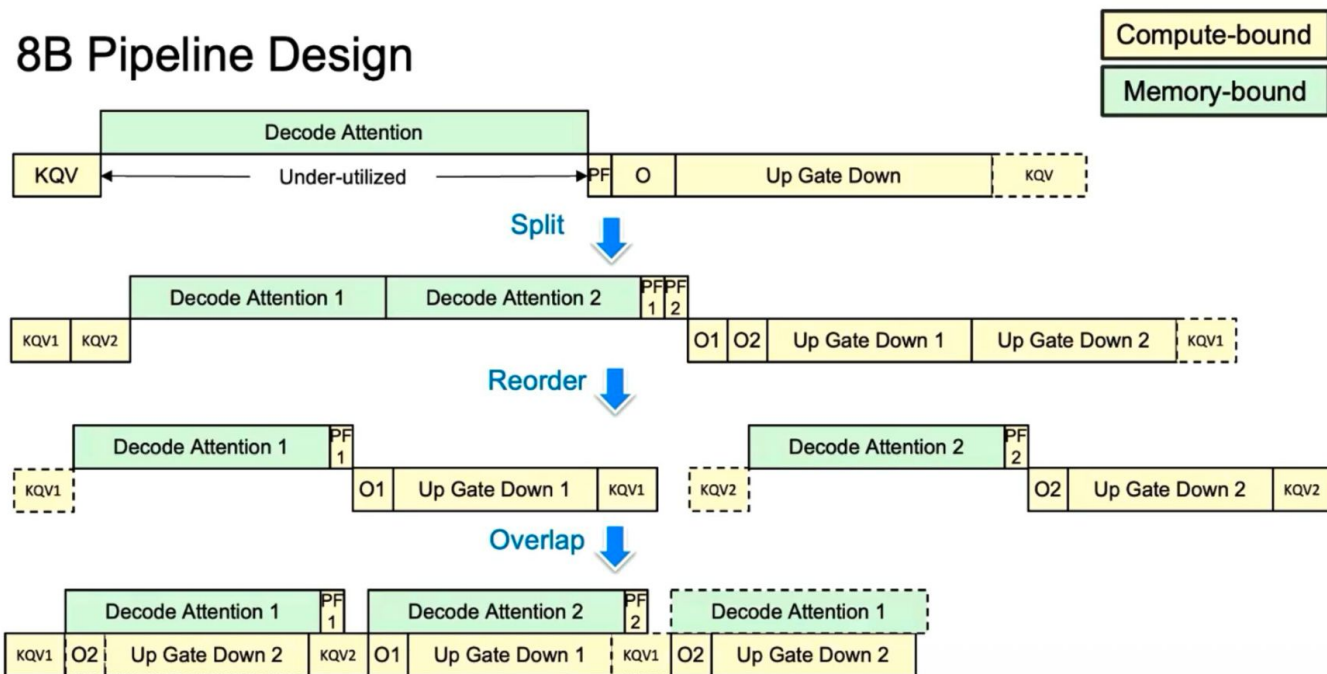
Stage I: Constraints

- Number of nano-operations
 - Split operations into nano-operations
- Batch size & runtime
 - Selected from profiling results
- Dependencies
- Overlap rules
 - Overlap operation with different resource constraints
- Operation transformations
 - Allow equivalent collectives ($AG \leftrightarrow AR$)



Design Example:

8B Pipeline Design





Stage II: Refining the Pipeline

- **Goal:**
 - Minimize pipeline time considering interference

Inputs

Execution Time Map $Op, B, R \rightarrow t$
Dependency DAG
NanoBatch Num
NanoBatch Size
NanoOp Ordering

Mixed Integer Linear
Programming

Outputs

Resource Allocation R

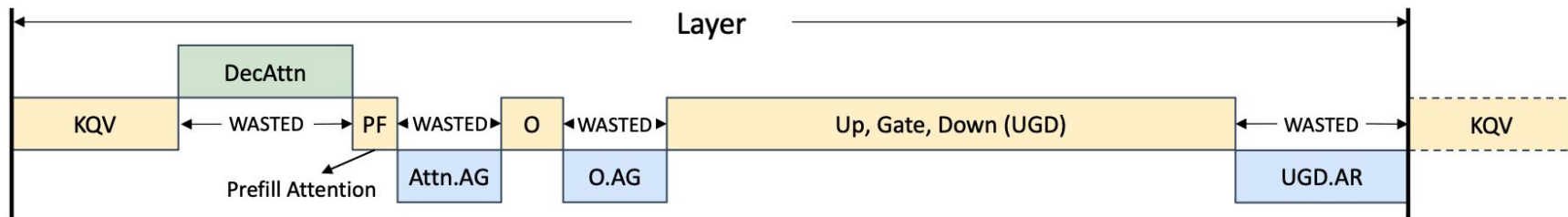


Stage II: Constraints

- Pipeline structure
 - Ordering and batch sizes fixed from Stage I
- GPU resource utilization
 - Sum of resource shares $\sum \mathbf{R} \leq 1.0$ at all times
- Execution time
 - Runtime = D_{best} / P
 - D_{best} : best execution time without interference



Example Pipeline:



Example Pipeline with NanoFlow:

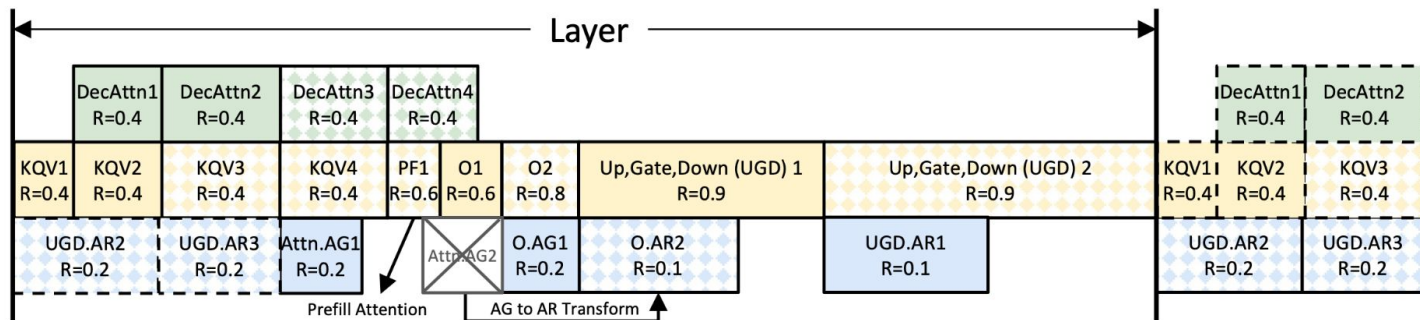


Figure 6: Execution pipeline of LLaMA-2 70B, automatically generated by NanoFlow. The solid background and shaded background represents input batch 0-768 and 768-2048, respectively. R stands for resource utilization. By overlapping the compute-, memory-, and network-intensive operations, NanoFlow increases compute utilization and improves the serving throughput.



NanoFlow Runtime

- Request Scheduling:
 - Batch formation
 - Asynchronous scheduling
- KV-cache Management:
 - Simultaneous offloading
 - Host KV-cache management
 - KV-cache loading and scattering



Request Scheduling

- Batch formation
 - Maintain a stable dense token batch for high GPU utilization
 - Prioritize unfinished decode requests
 - Fill remaining capacity using token-level prefill chunks



Request Scheduling (cont.)

- Asynchronous scheduling
 - Batch formation is CPU-intensive
 - Construct next batch while GPU executes current iteration
 - Finished requests removed one iteration later (negligible overhead)



KV-cache Management

- Simultaneous offloading
 - Offload KV-cache immediately after generation
 - Overlap transfers with computation to hide latency
- Host KV-cache management
 - Hierarchical cache across GPU, CPU memory, and SSD
 - LRU policy manages host-side KV-cache



KV-cache Management (cont.)

- Efficient loading
 - PagedAttention (2023) based GPU memory layout
 - Copy to contiguous GPU buffer, then scatter to pages
 - Achieves 7-10× higher bandwidth for host-to-device copy



MICHIGAN ENGINEERING
UNIVERSITY OF MICHIGAN

NanoFlow Evaluation



NanoFlow Experiments

- Throughput
- Latency
- Ablation Study
- Resource Usage
- Performance of Other LLMs



NanoFlow Experiment Setup

- Hardware:
 - 8 × A100 90GB SXM GPUs interconnected via NVLink
- Model:
 - LLaMA-2-70B model
- Baselines:
 - vLLM
 - DeepSpeed-FastGen
 - TensorRT-LLM

- Workloads:

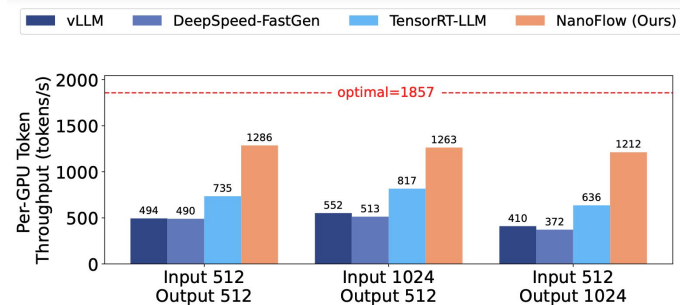
Dataset	Avg. Input (Std)	Avg. Output (Std)
Splitwise [32]	1155 (1109)	211 (163)
LMSYS-Chat [56]	102 (169)	222 (210)
ShareGPT [1]	246 (547)	322 (244)

Table 4: The average and standard deviation of input and output lengths in the sampled datasets.

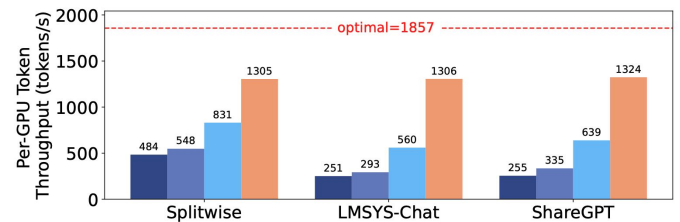


NanoFlow Throughput

- Experiment:
 - Real workloads sampled from conversational datasets
 - Additional fixed-length workloads for controlled comparison



(a) LLaMA-2-70B, 8 GPU, TP=8, Constant Input & Output Length

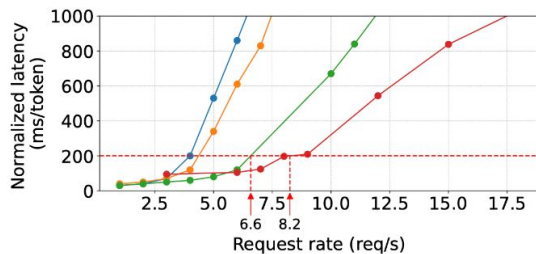


(b) LLaMA-2-70B, 8 GPU, TP=8, Input & Output Length from Dataset

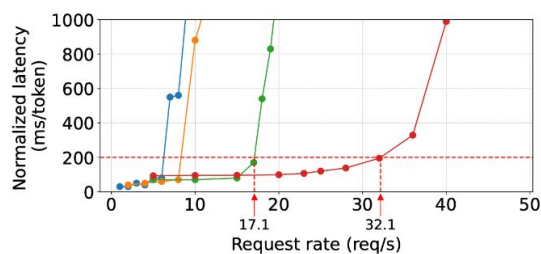
NanoFlow Latency

- Experiment:
 - Request traces generated from real datasets
 - Gradually increase request arrival rate
 - Measure normalized end-to-end latency

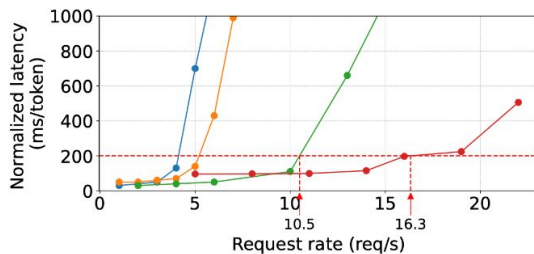
— vLLM — DeepSpeed-FastGen — TensorRT-LLM — NanoFlow (Ours)



(a) Splitwise



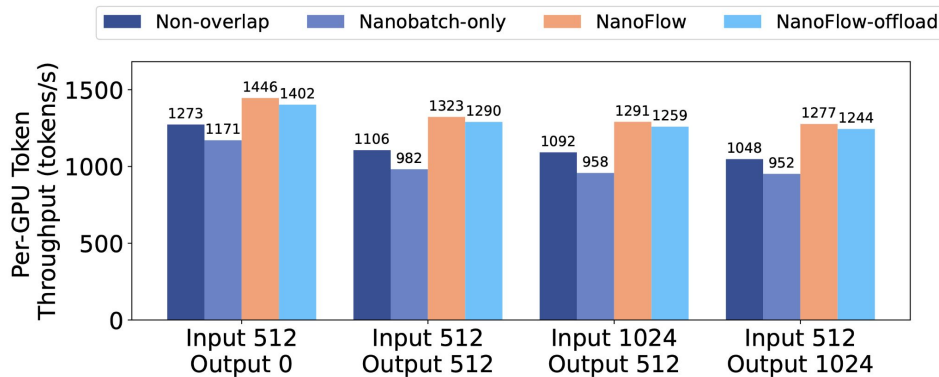
(b) LMSYS-Chat-1M



(c) ShareGPT

NanoFlow Ablation Study

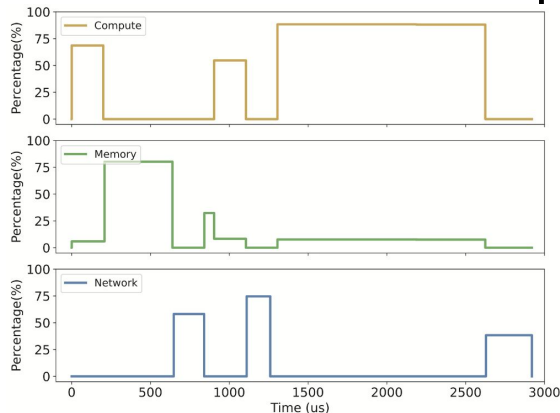
- Experiment:
 - Sequential execution (no nano-batching, no overlap)
 - Nano-batching only (no overlap)
 - NanoFlow-offload (enable KV-cache offloading)



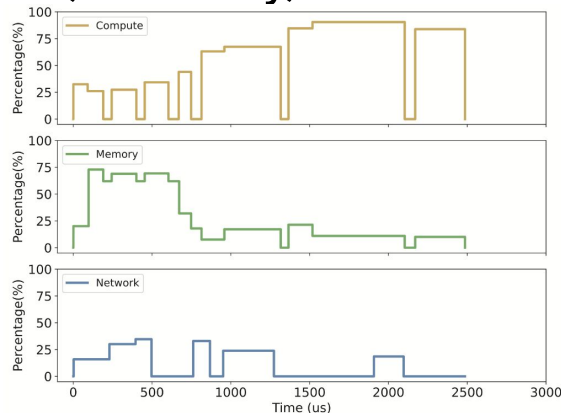


NanoFlow Resource Usage

- Key observation:
 - NanoFlow achieves higher and more balanced utilization across compute, memory, and network



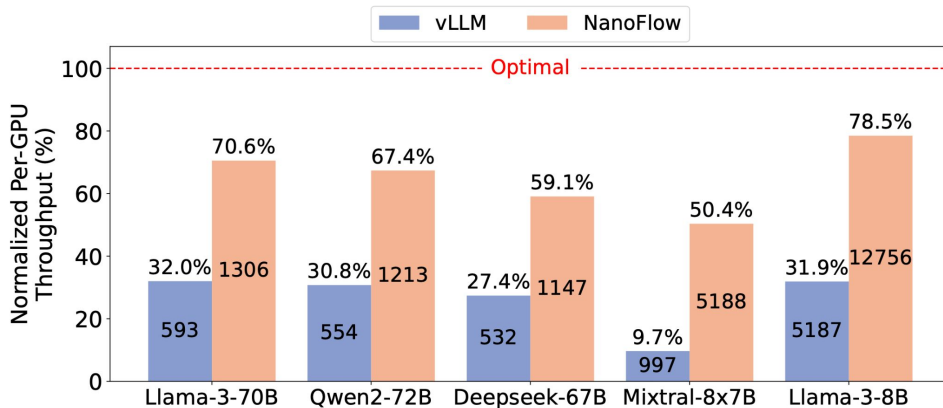
(a) Non-overlap pipeline resource usage



(b) NanoFlow resource usage

NanoFlow Performance on LLMs

- Experiment:
 - Evaluate throughput with constant length of input 1024 and output 512





MICHIGAN ENGINEERING
UNIVERSITY OF MICHIGAN

MOONCAKE (2025)

**Trading More Storage for Less Computation -
A KVCache-centric Architecture for Serving LLM
Chatbot**



More Storage for Less Computation

Past Works

- Separation of prefill and decode stages
- Local caching of KVCache from prefill stages

Kimi: goal is to maximize overall throughput for a large-scale LLM for large-requests

- Longer request context → larger KVCache
- Require a lot more storage!!!



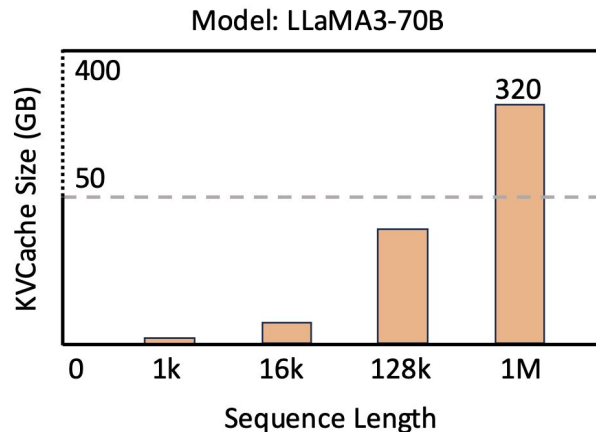
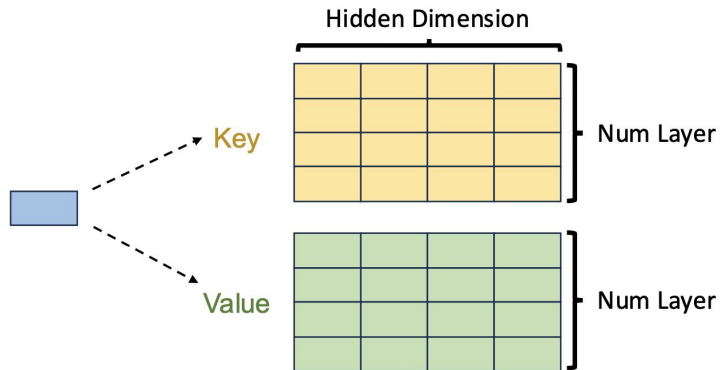
Challenge to Store Large KVCache

One token KVCache is very large

KV Cache linearly increase with sequence length

One token (Bytes)

One token's KVCache (Tens of KB)





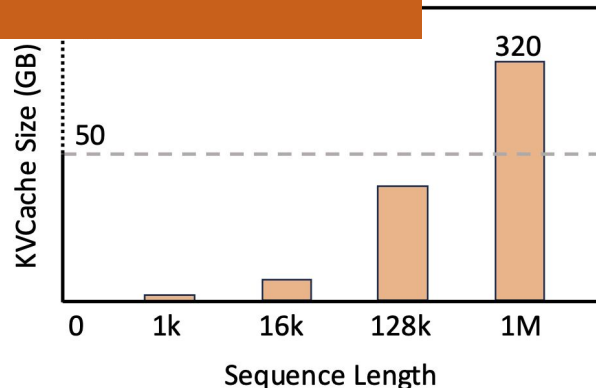
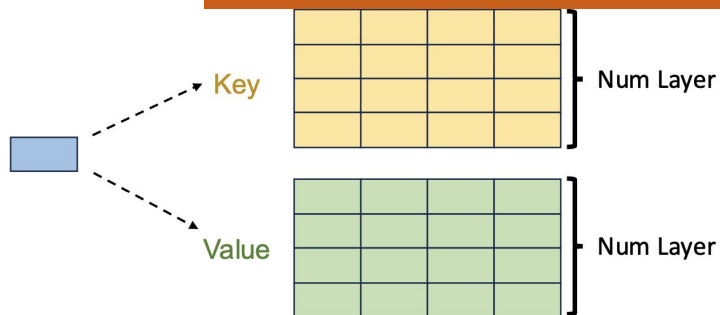
Challenge to Store Large KVCache

One token KVCache is very large

KV Cache linearly increase with

One token (Bytes)

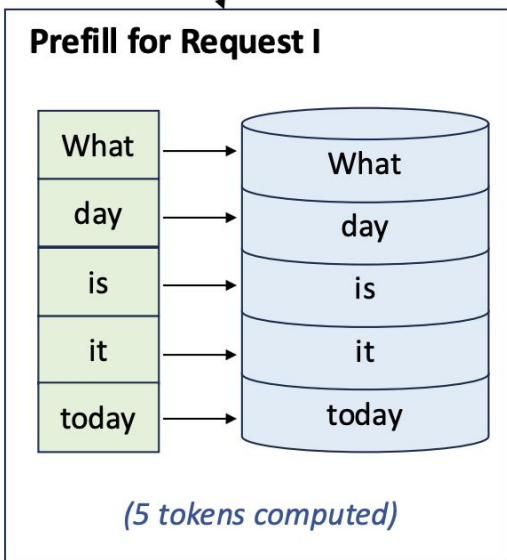
Solution: Global Cache!





Global cache can reduce prefill computation

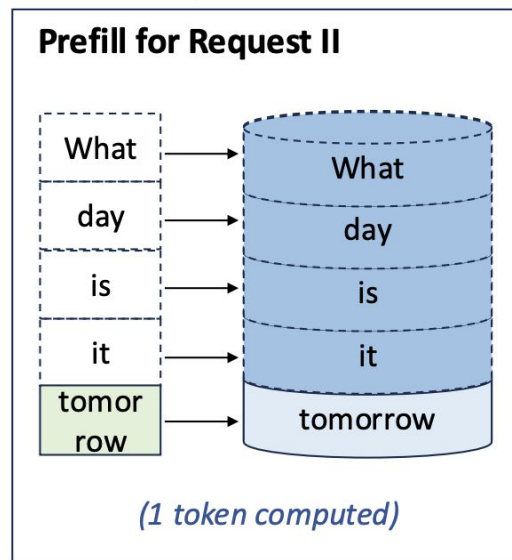
"What day is it today"



KVCache Reuse



"What day is it tomorrow"





MOONCAKE

KVCache-centric global cache and scheduler with disaggregated prefill and decode instances

1. **MOONCAKE Store:** manages storage and transfer of global KVCache blocks
2. **Conductor:** global scheduler responsible for dispatching requests based on current distribution of KVCache and workload

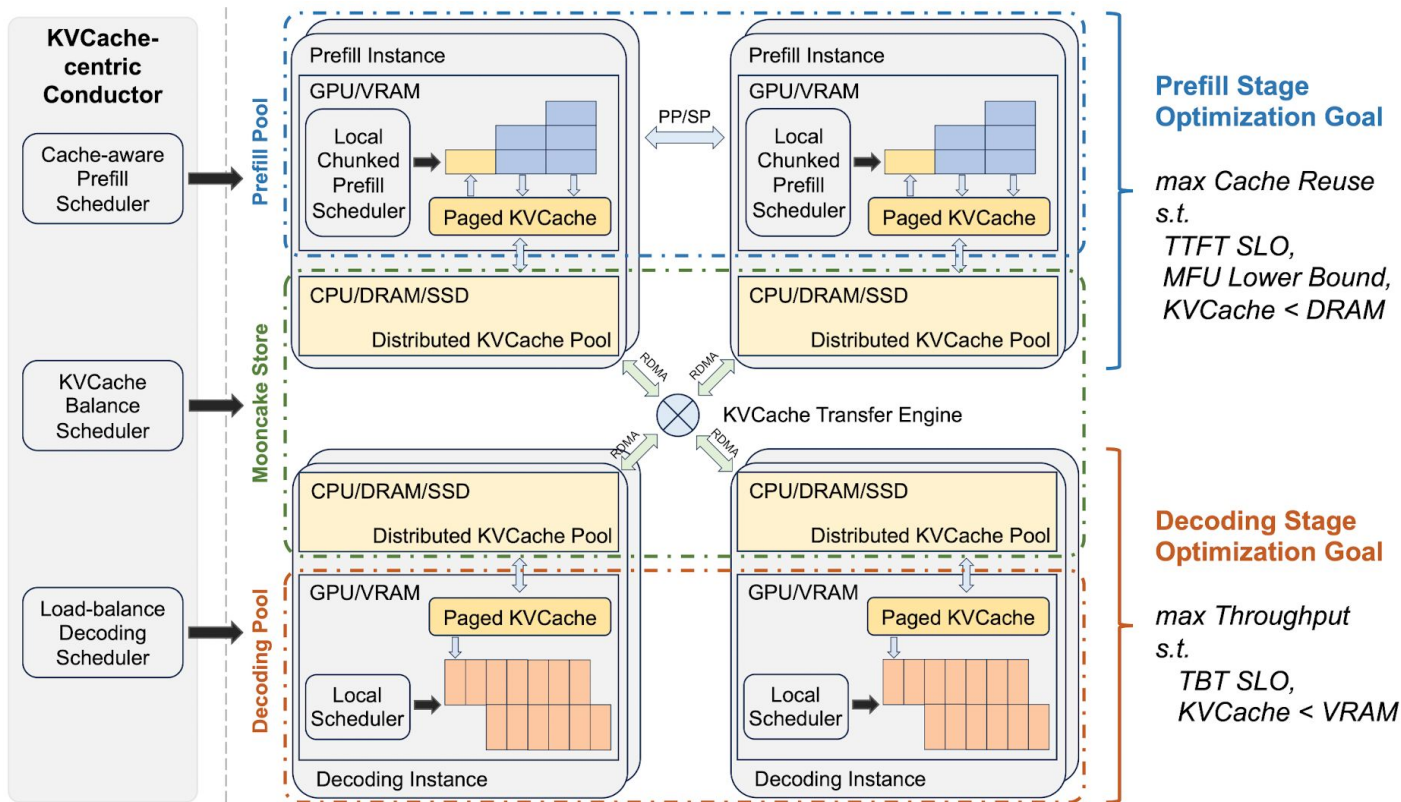


Figure 2: MOONCAKE Architecture.

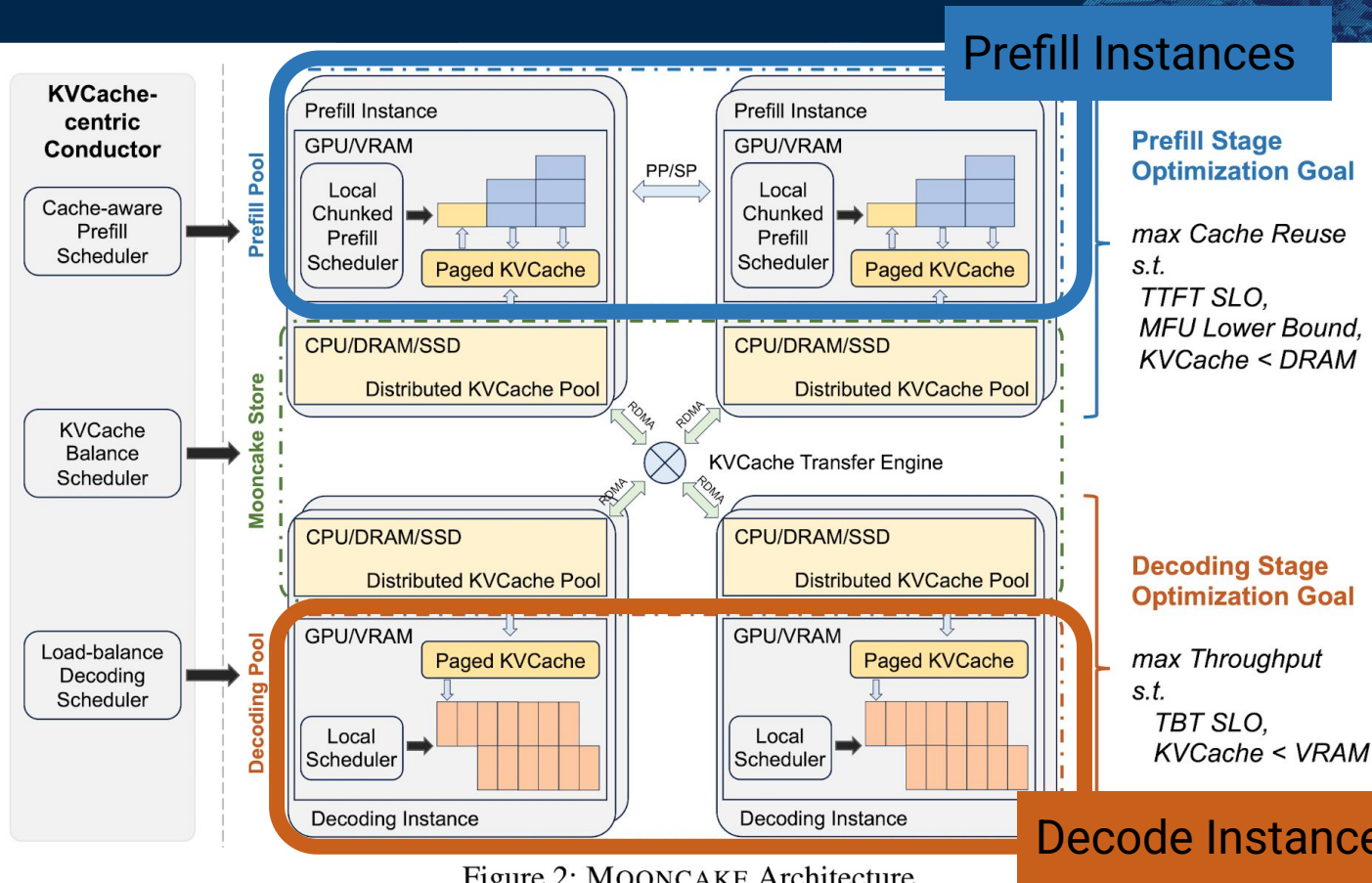


Figure 2: MOONCAKE Architecture.

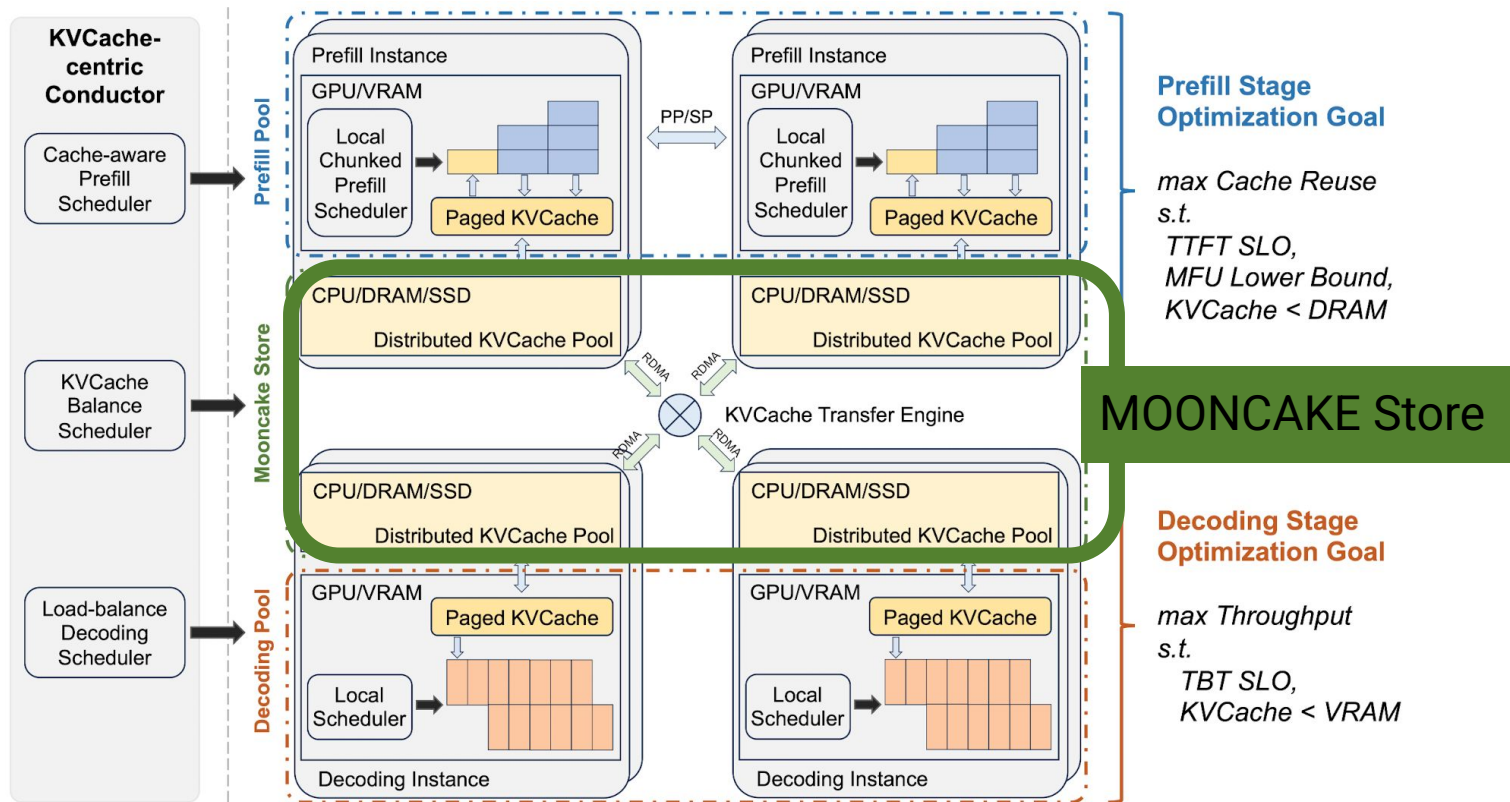


Figure 2: MOONCAKE Architecture.

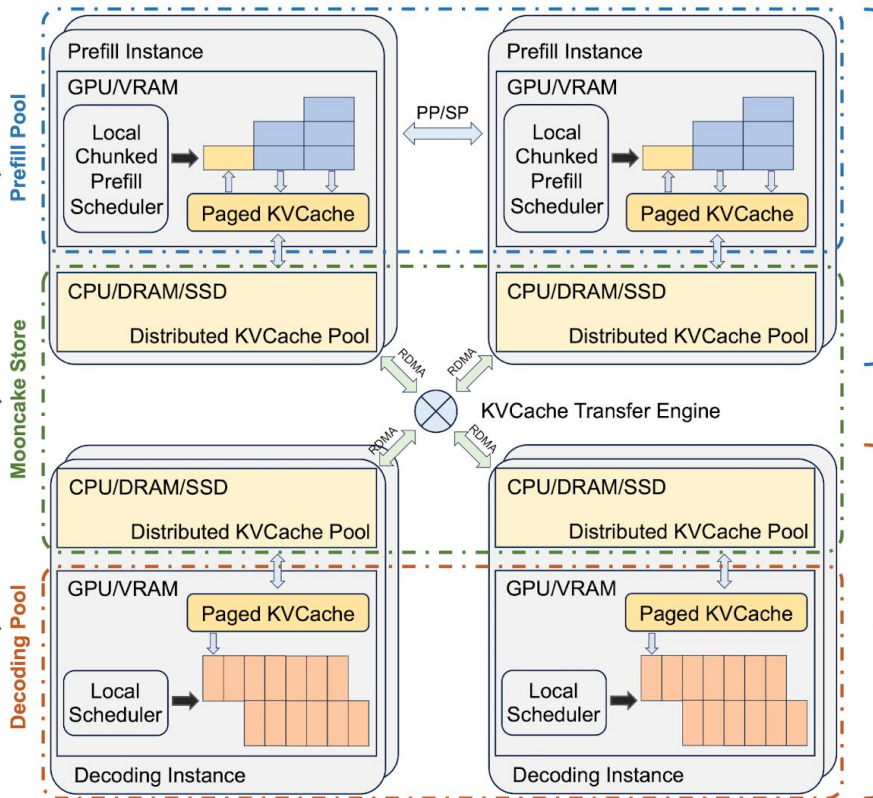
Conductor

KVCache-centric Conductor

Cache-aware Prefill Scheduler

KVCache Balance Scheduler

Load-balance Decoding Scheduler



Prefill Stage Optimization Goal

max Cache Reuse
s.t.
TTFT SLO,
MFU Lower Bound,
KVCache < DRAM

Decoding Stage Optimization Goal

max Throughput
s.t.
TBT SLO,
KVCache < VRAM

Figure 2: MOONCAKE Architecture.

Inference Workflow

1. **KVCache Reuse:** reusable KVCache blocks transferred to prefill instance
2. **Incremental prefill:** Prefill stage
3. **KVCache Transfer:** Stream generated KVCache to designated decoding instance
4. **Decoding:** decoding request joins next batch

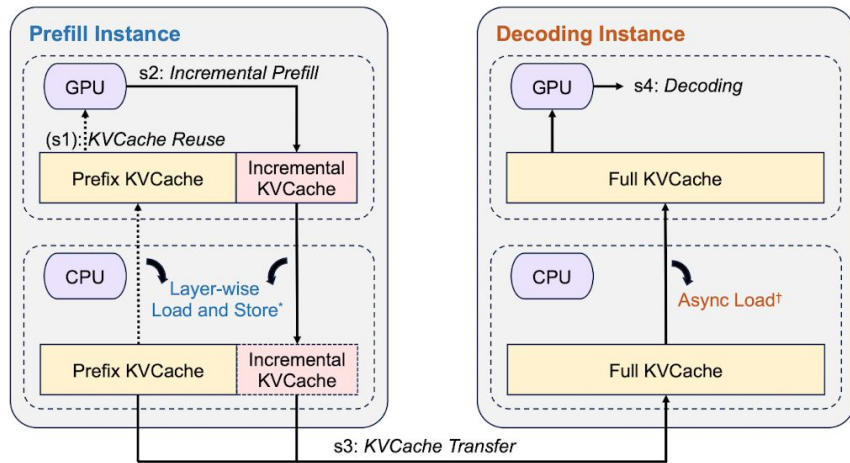


Figure 3: Workflow of inference instances.



Prefix Cache Blocks



Incremental Cache Blocks



Unallocated Cache Blocks

Token Blocks
(16~512 tokens)

a	b	c	d	e	f	g	h	i
---	---	---	---	---	---	---	---	---

$A = \text{Hash}(a)$	$B = \text{Hash}(A+b)$	$C = \text{Hash}(B+c)$	$D = \text{Hash}(C+d)$	$E = \text{Hash}(D+e)$

Prefill
Instance

Decoding
Instance



Prefix Cache Blocks

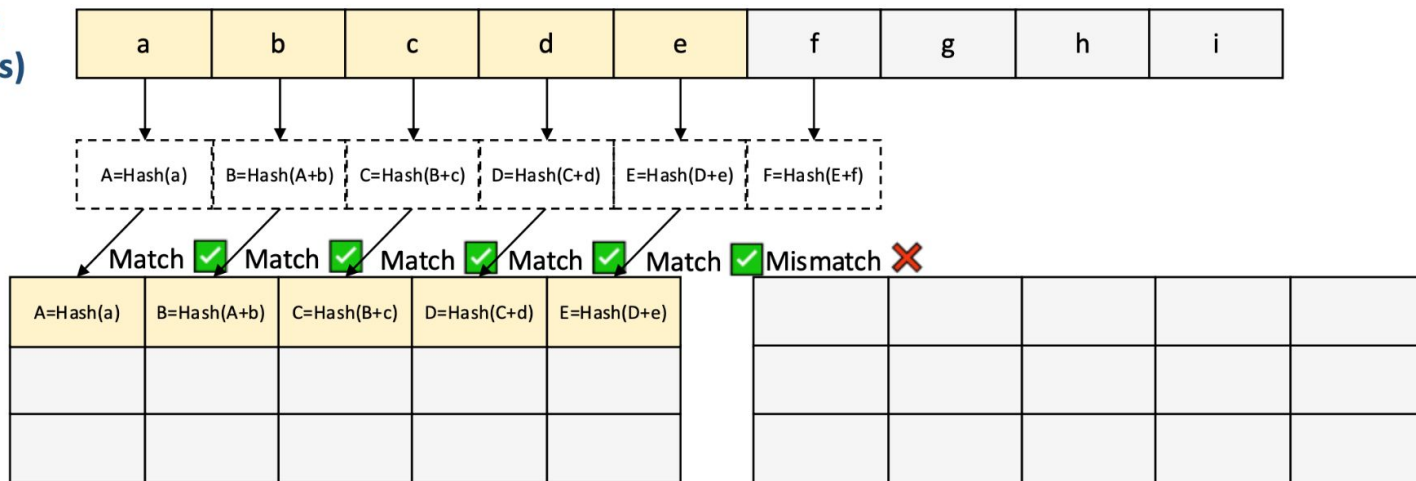


Incremental Cache Blocks



Unallocated Cache Blocks

Token Blocks
(16~512 tokens)



Prefill
Instance

Decoding
Instance



Prefix Cache Blocks

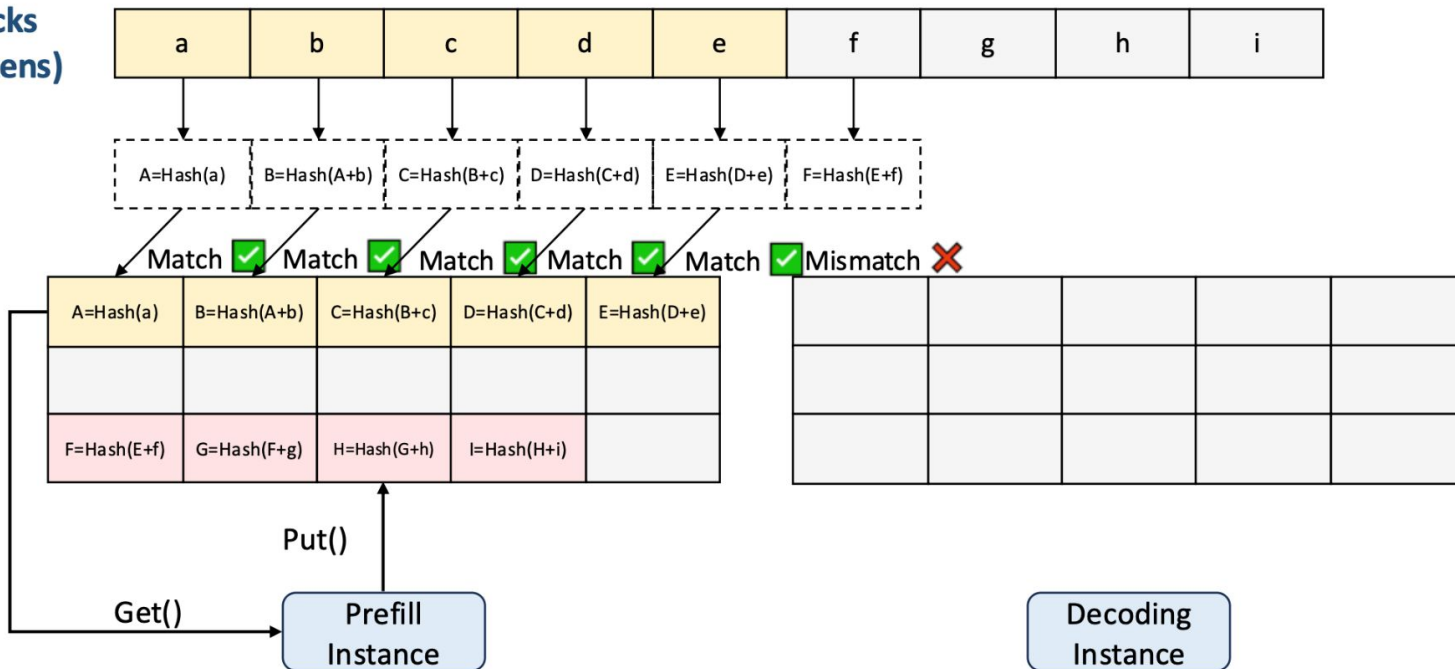


Incremental Cache Blocks



Unallocated Cache Blocks

Token Blocks
(16~512 tokens)





Prefix Cache Blocks

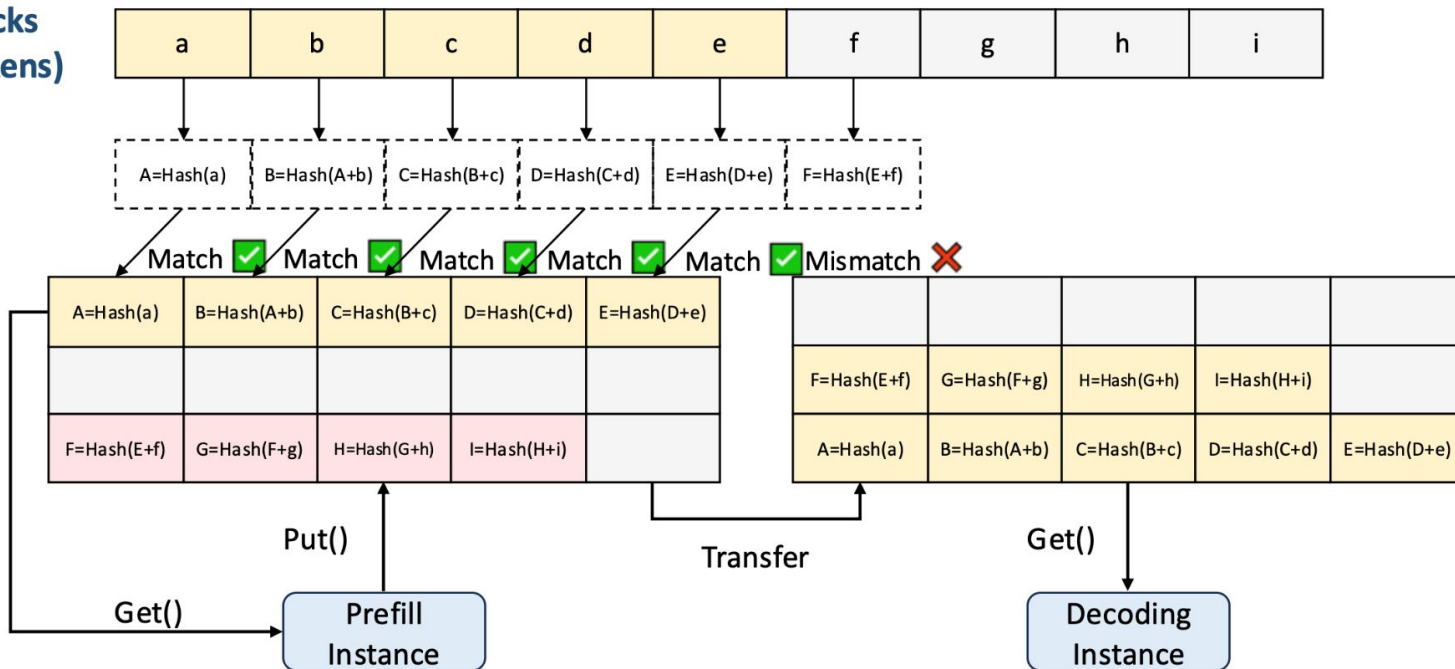


Incremental Cache Blocks



Unallocated Cache Blocks

Token Blocks
(16~512 tokens)



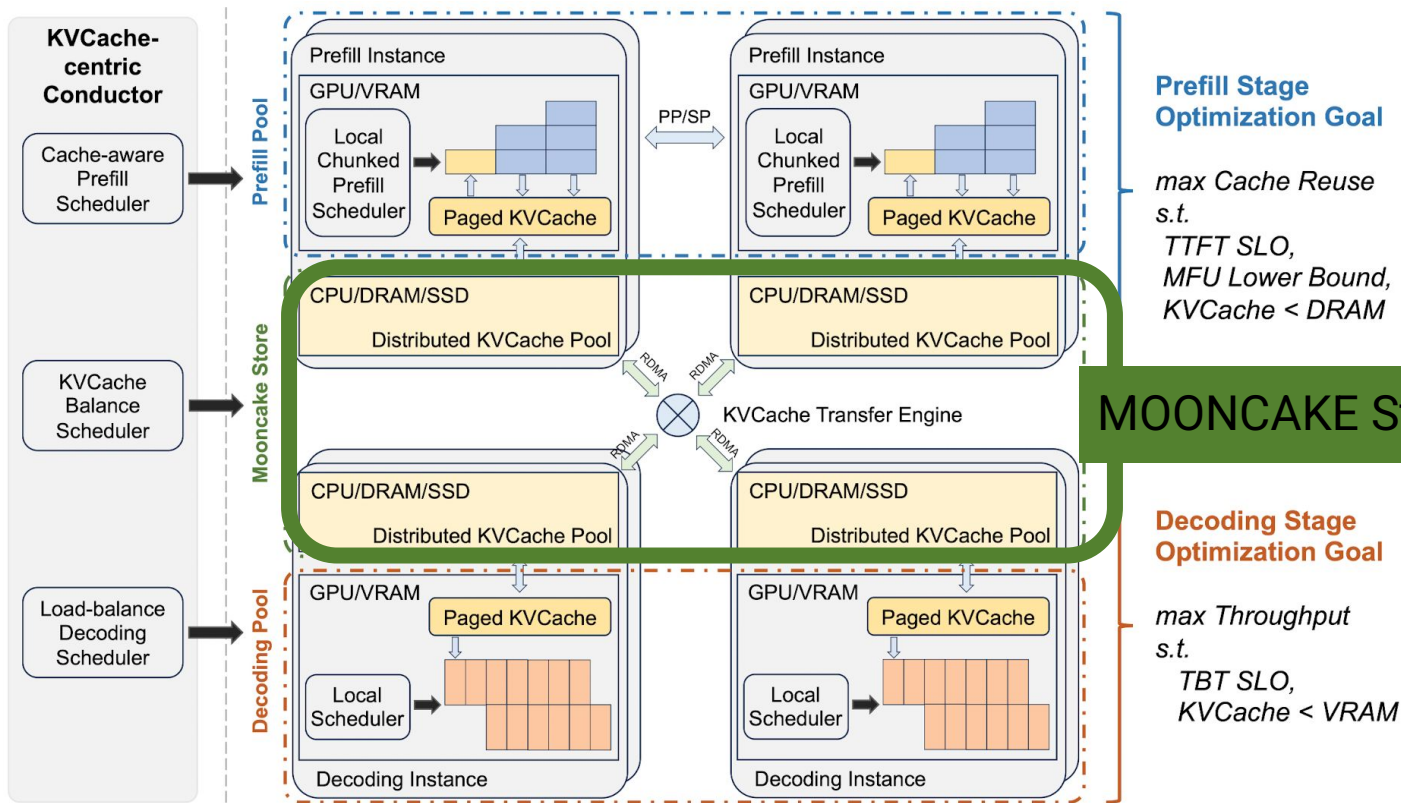


Figure 2: MOONCAKE Architecture.



1. MOONCAKE Store

MOONCAKE Store is a distributed global cache of KVCache

- KVCache stored as paged blocks within a distributed cache pool
- Offers object-based API to facilitate KVCaching

Listing 1: Memory transfer APIs in MOONCAKE Store.

```
int registerLocalMemory(void *vaddr, size_t len,  
                        const string &type);  
BatchID allocateBatchID(size_t batch_size);  
int submitTransfer(BatchID batch_id,  
                  const vector<Request> &entries);  
int getTransferStatus(BatchID batch_id,  
                     int request_index,  
                     Status &status);  
int freeBatchID(BatchID batch_id);
```

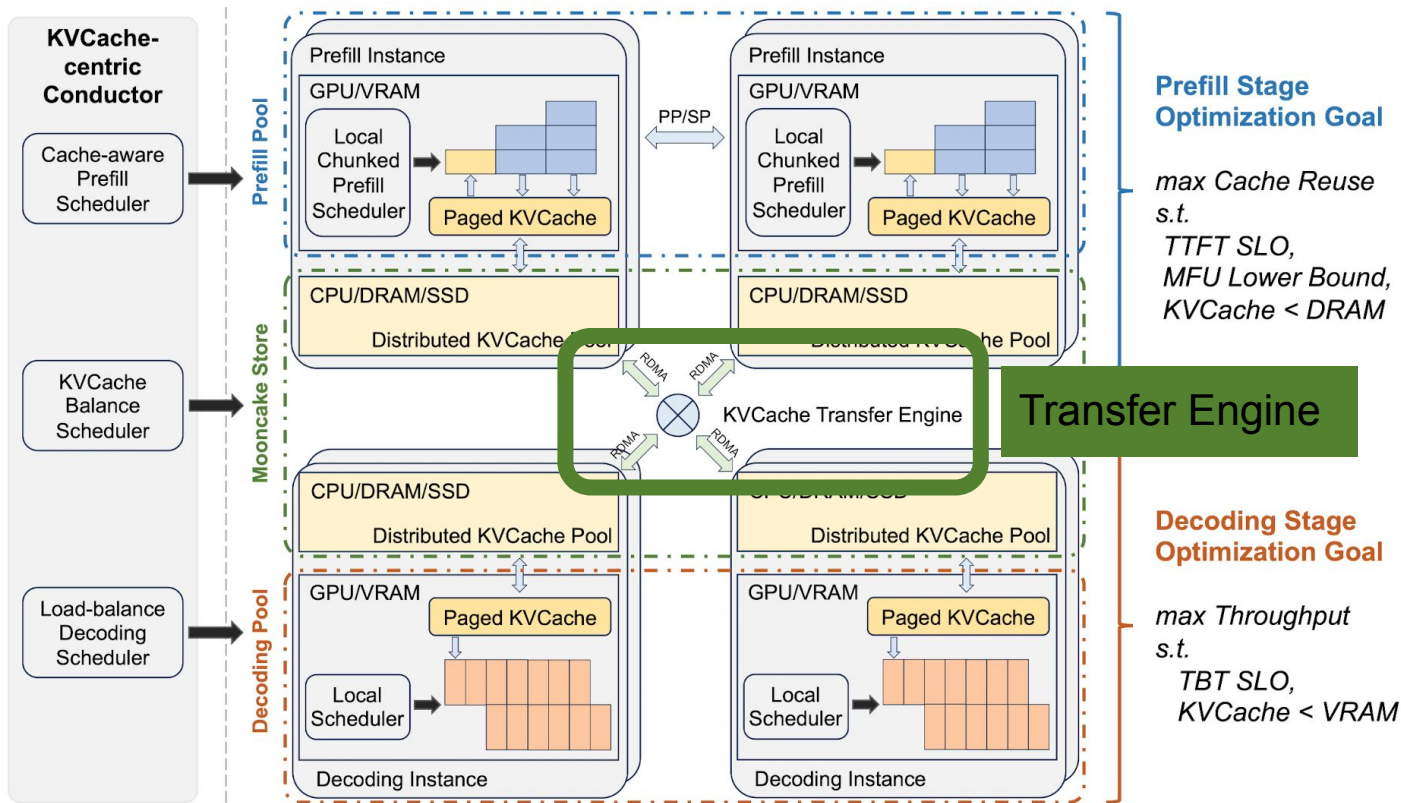


Figure 2: MOONCAKE Architecture.



Transfer Engine

Designed to effectively transfer KVCache between prefill and decode instances and throughout MOONCAKE Store

- **Network setup:** Mooncake relies on high-bandwidth network interconnect → utilizes RoCEv2
- **Topology Matrix:** categorizes Network Interface Cards (NICs) into preferred and secondary
- **Endpoint management:** connection between instances are managed as “endpoints” that are managed in a pool

Topology-aware path selection enables efficient connection and execution of data transfer

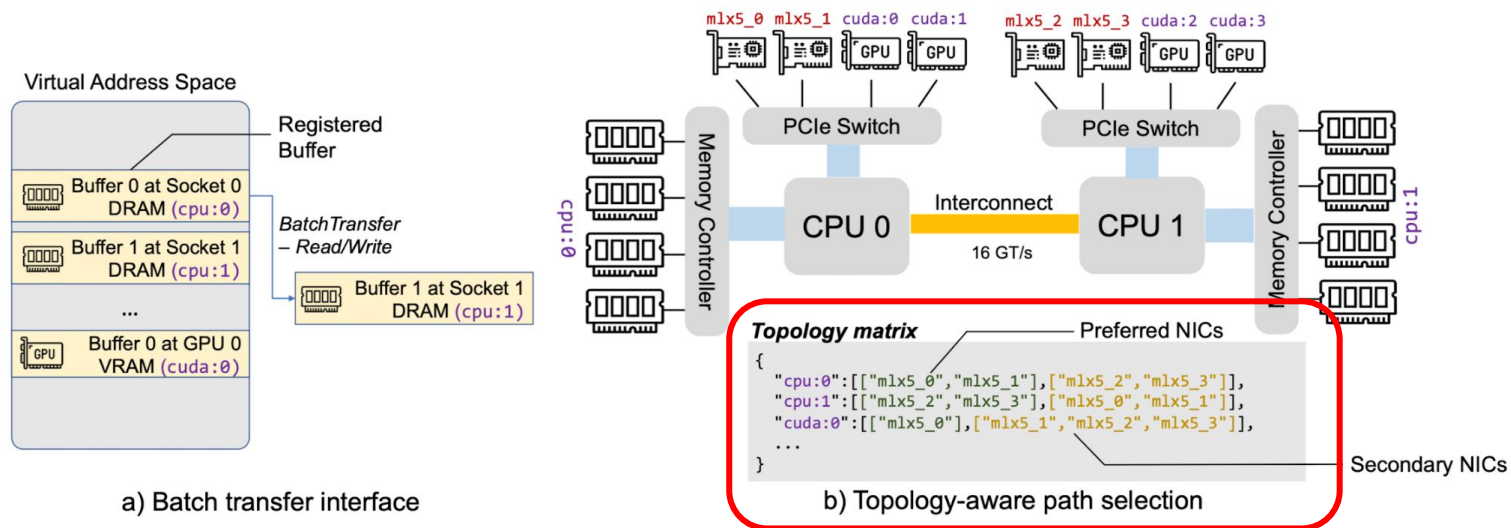


Figure 4: Transfer engine of MOONCAKE Store.



MOONCAKE Store Prefill Pool

Disaggregated pool of prefill instances handle incoming requests

- **Handle Overload:** Conductor decides which prefill instance to send request to based on computation overhead
- **Chunked Pipeline Parallelism (CPP):** request context input is partitioned into chunks that are processed across multiple instances in parallel

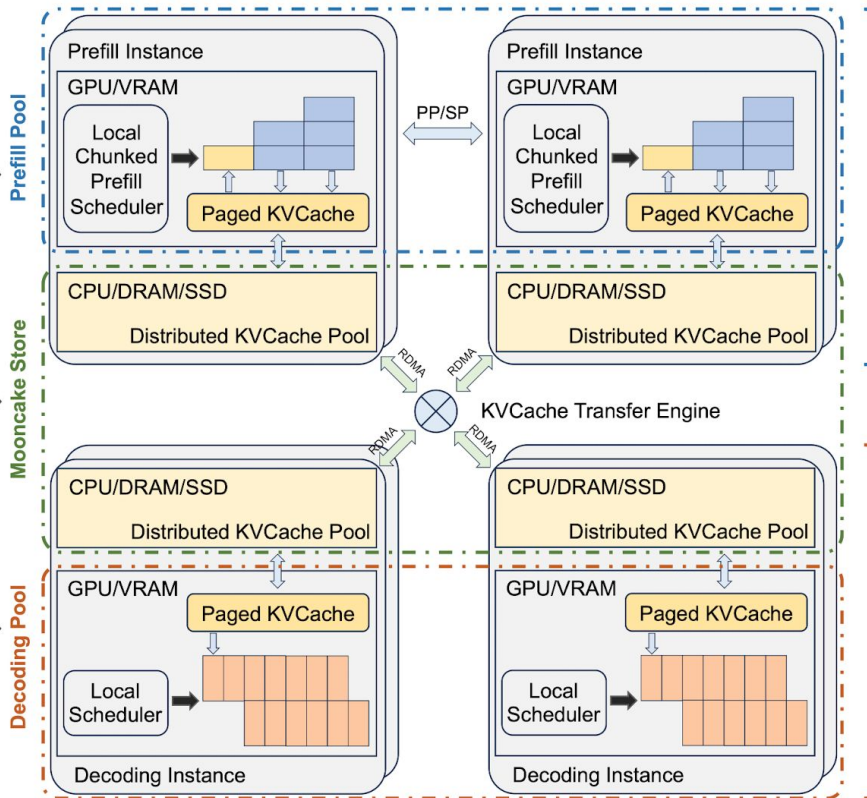
Conductor

KVCache-centric Conductor

Cache-aware Prefill Scheduler

KVCache Balance Scheduler

Load-balance Decoding Scheduler



Prefill Stage Optimization Goal

max Cache Reuse
s.t.
TTFT SLO,
MFU Lower Bound,
KVCache < DRAM

Decoding Stage Optimization Goal

max Throughput
s.t.
TBT SLO,
KVCache < VRAM

Figure 2: MOONCAKE Architecture.



2. Conductor: Global Scheduler

Global scheduler goal is

- **Prefill Stage:** reuse as much KVCache as possible
- **Decode Stage:** aggregate as many tokens as possible to improve MFU time and meet TBT

How does conductor pick best prefill and decode instances for a request?

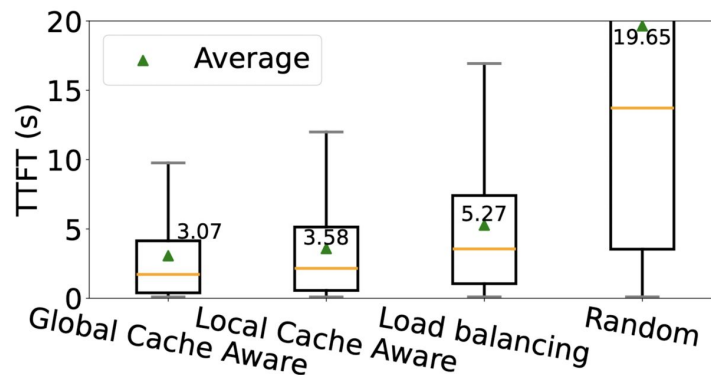


Figure 5: The prefill scheduling experiment.



Scheduling Algorithm

1. Hash request prefill context
2. Find instance with best global prefix match
3. For each node in prefill pool
 - a. If global prefix match is much longer than current prefix match, estimate **time to transfer** KVCache from best global prefix match
 - b. Else, **time to transfer** is 0 (recompute cache on current prefill instance)
 - c. Estimate **prefill queue time** and **prefill execution time**
4. Select prefill instance with shortest TTFT

$$TTFT \leftarrow T_{transfer} + T_{queue} + T_{prefill}$$



How to Estimate Prefill Time?

Estimate Prefill Queue Time

- Aggregate the prefill times of all queued requests for current instance

Estimate Prefill Execution Time

- Based on request length
- Polynomial regression model fitted with offline data

TTFTs are computed in parallel → estimation processing time is negligible compared to inference time



Cache Load Balancing

Some requests will not always be sent to prefill instance with longest prefix cache match due to high instance load

- **Is it faster to transfer KVCache from best match or recompute tokens?**
 - Balancing threshold is set to determine this threshold
 - Lower threshold means that popular caches are more likely to be replicated across prefill instance

Reduce prefill time and distribute hotspot caches!



MICHIGAN ENGINEERING
UNIVERSITY OF MICHIGAN

MOONCAKE Evaluation



Evaluation

1. Does MOONCAKE outperform existing LLM inference systems in real-world scenarios?
2. Compared to conventional prefix caching methods, does the design on MOONCAKE Store significantly improve MOONCAKE's performance?



Evaluation

1. Does MOONCAKE outperform existing LLM inference systems in real-world scenarios?



Setup

- Evaluated on a dummy LLaMA3-70B model
- Baseline
 - **vLLM**: state of the art serving system, which does not separate prefill and decode into different nodes
 - **vLLM with chunked prefill**
 - **vLLM with prefix cache** (local to each node)
- Workloads
 - Conversation, Tool & Agent, Synthetic
- **Service Level Objectives (SLOs)**: metrics that define good service
 - Time to First Token(TTFT)
 - Time between Token (TBT)



Workloads

Cache ratio: % of prompt tokens that can be served from KV cache during prefill

	Conversation	Tool&Agent	Synthetic
Avg Input Len	12035	8596	15325
Avg Output Len	343	182	149
Cache Ratio	40%	59%	66%
Arrival Pattern	Timestamp	Timestamp	Poisson
Num Requests	12031	23608	3993

Table 2: Workload Statistics.



Workloads

Cache ratio: % of prompt tokens that can be served from KV cache during prefill

	Conversation	Tool&Agent	Synthetic
Avg Input Len	12035	8596	15325
Avg Output Len	343	182	149
Cache Ratio	40%	59%	66%
Arrival Pattern	Timestamp	Timestamp	Poisson
Num Requests	12031	23608	3993

Table 2: Workload Statistics.



Workloads

Cache ratio: % of prompt tokens that can be served from KV cache during prefill

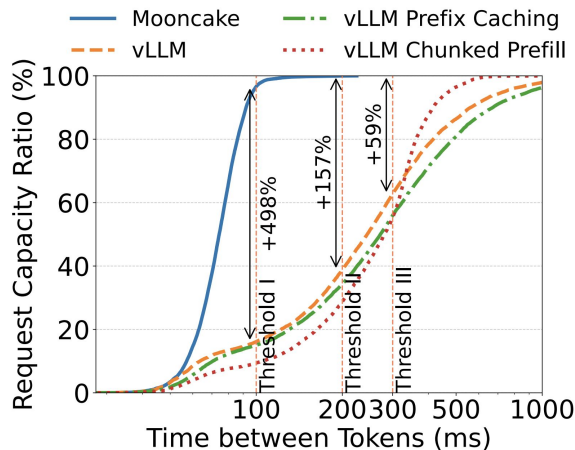
	Conversation	Tool&Agent	Synthetic
Avg Input Len	12035	8596	15325
Avg Output Len	343	182	149
Cache Ratio	40%	59%	66%
Arrival Pattern	Timestamp	Timestamp	Poisson
Num Requests	12031	23608	3993

Table 2: Workload Statistics.

Effective Request Capacity

Effective Request Capacity: % of incoming requests that the system can successfully handle while still meeting its SLOs

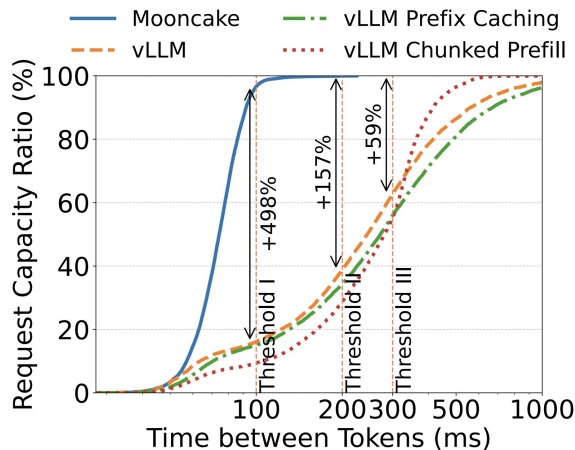
Conversation



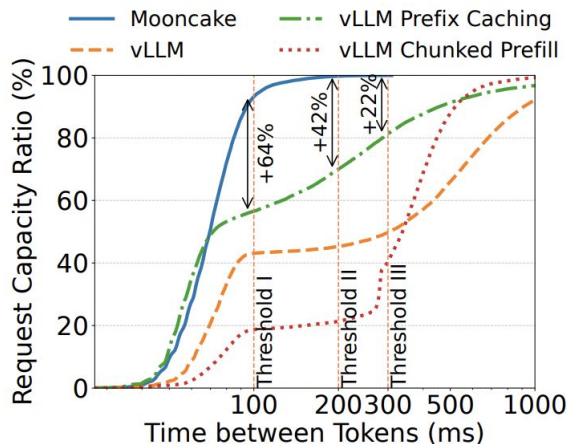
Effective Request Capacity

Effective Request Capacity: % of incoming requests that the system can successfully handle while still meeting its SLOs

Conversation



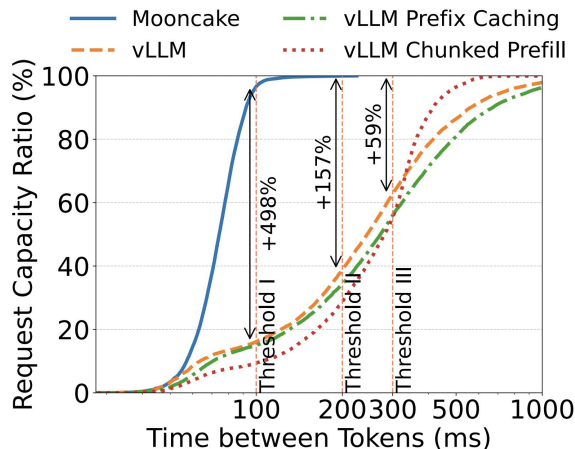
Tool & Agent



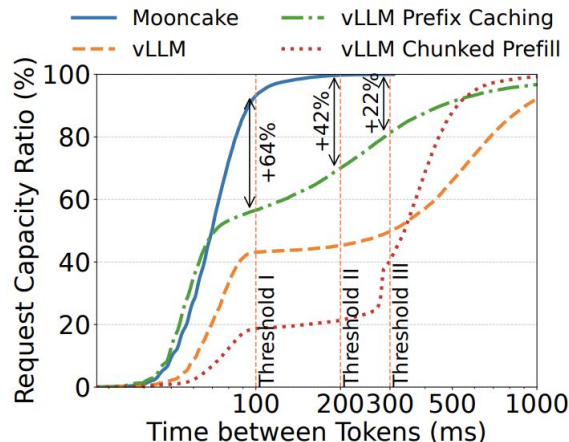
Effective Request Capacity

Effective Request Capacity: % of incoming requests that the system can successfully handle while still meeting its SLOs

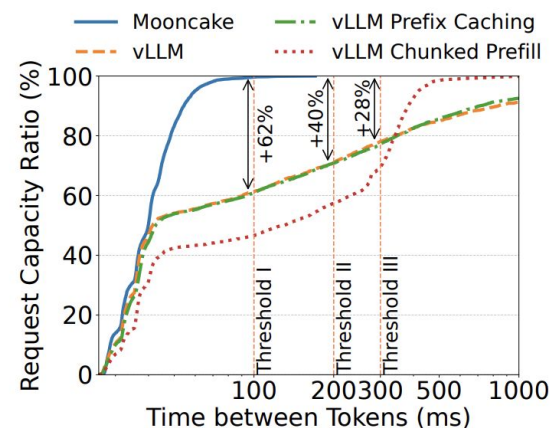
Conversation



Tool & Agent



Synthetic



Prefill GPU Time

- Motivation: Prefill GPU time is positively correlated with requests' TTFT and serving costs
- Mooncake exhibits significantly less prefill GPU time

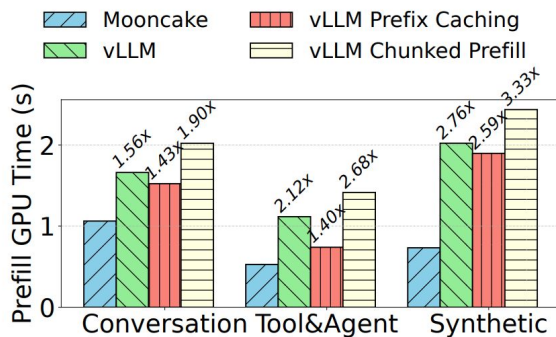


Figure 8: Average GPU time of each request during the prefill stage under different workloads.

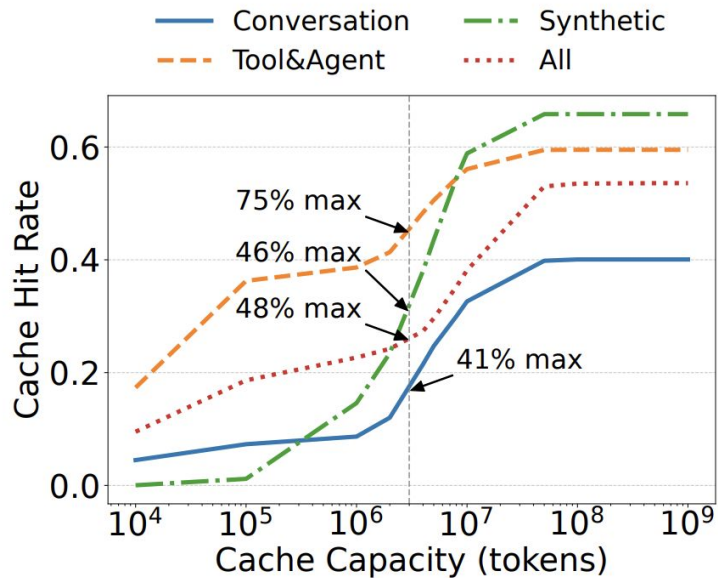


Evaluation

2. Compared to conventional prefix caching methods, does the design on MOONCAKE Store significantly improve MOONCAKE's performance?

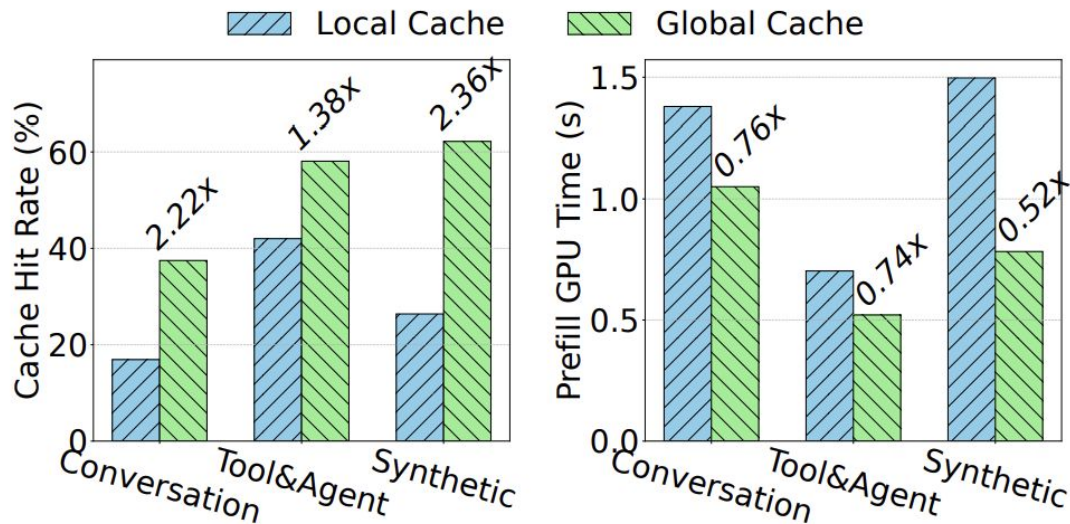


Impact of Cache Capacity



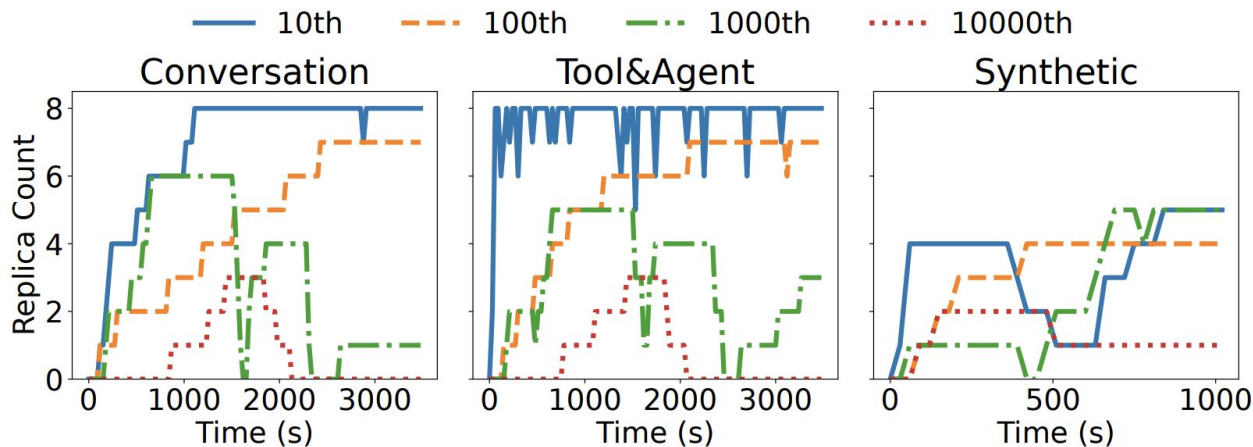


Impact of Global Cache



Replicas

- Recorded the cumulative access counts of all cache blocks and ranked them





KV Transfer Engine Performance

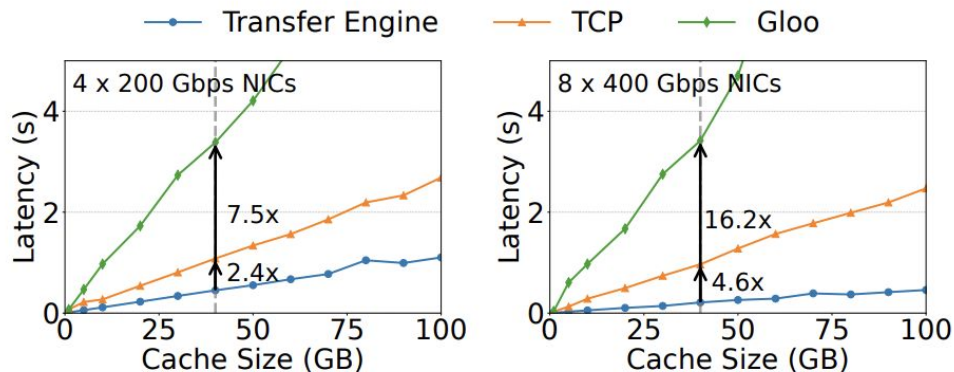
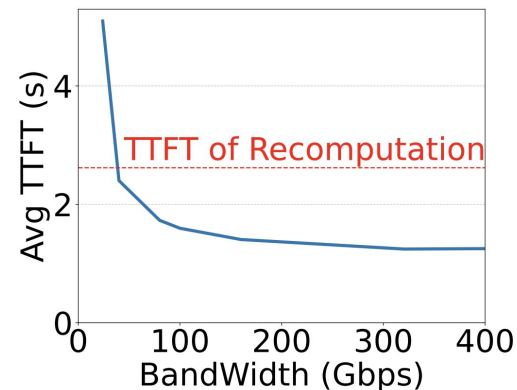
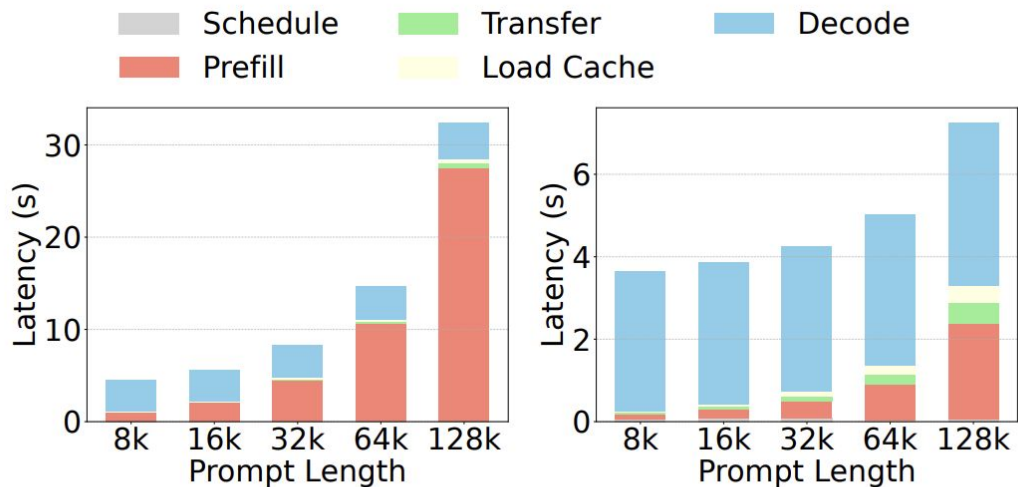


Figure 12: Latency of inter-node cache transfer.





End-to-End Latency Breakdown



(a) Prefix cache ratio 0%.

(b) Prefix cache ratio 95%.



MICHIGAN ENGINEERING
UNIVERSITY OF MICHIGAN

Conclusion + Thoughts



Comparisons

	Nanoflow	Mooncake
Scope	Intra-node	Inter-node
Disaggregation of nodes (prefill & decode)	A single node is allowed to perform both prefill and decode operations	Disaggregation of prefill and decode nodes



Challenges and Limitations

1. Nanoflow
 - a. Significant improvements for latency only occur at high request rates
 - b. Nano-batches and overlap do not seem to provide the majority of the performance increase
2. Mooncake
 - a. Requires high bandwidth
 - i. >100 Gbps to ensure optimal system performance
 - b. Demonstrated results only over 3 types of workloads
 - i. All three workloads had high input lengths



Future Works

1. Nanoflow
 - a. Dynamic scheduling based on request rate
2. Mooncake
 - a. More custom and efficient cache design and eviction policies
 - b. SLO differentiation across different kinds of workloads
 - c. More flexible way to compute cache balancing threshold based on cost of recomputation or other factors



MICHIGAN ENGINEERING
UNIVERSITY OF MICHIGAN

Thank you!



MICHIGAN ENGINEERING
UNIVERSITY OF MICHIGAN

Q&A!

Things to cover:

- Background (5min) (Alan)
 - Prefill and Decode
 - Bounding Operations
 - roofline model
 - KV cache
 - Important operations
- NanoFlow (10)
 - Analysis of Compute-bound (motivates nanoflow)
 - Nano-flow design (Evan)
 - Nano batches
 - Auto-search algorithm
 - Experiments and Evaluation
- Mooncake (10) (Madeleine)
 - Motivations/Intro
 - Mooncake Design
 - Disaggregation of Prefill and Decode (mooncake store)
 - Transfer Engine (network)
 - Chunked Prefill pipeline
 - Scheduling (done by conductor)
 - Cache load balancing
 - Experiments and Evaluation (frank)
- Paper Comparison (2)
 - Mooncake operates on global level (disaggregation of nodes from the beginning)
 - Nanoflow operates on per GPU scale, parallelizing operations on the intra-device level
- Future Work and Weaknesses (3)

Things to keep in mind

- Implementation Fairness
- Parameter Tuning: was everything tuned correctly
- Model Selection: do the models reflect current state-of-the-art
- Workload diversity
- Practical latency: connection between reported metrics and absolute user-experienced latency
- Metric selection
- Performance distribution
- No obscuring normalization
-

Evaluating LLM Inference Systems

- Implementation Fairness
 - DistServe vs NanoFlow
 - Are baselines implemented comparably?
 - If not, are microbenchmarks or ablations used to isolate algorithmic gains from system implementation differences?

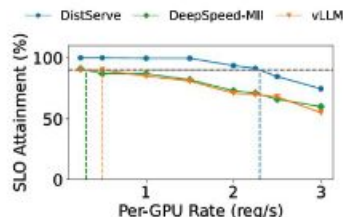


Figure 8: Chatbot application with OPT models on the ShareGPT dataset.

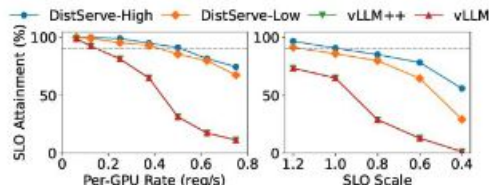
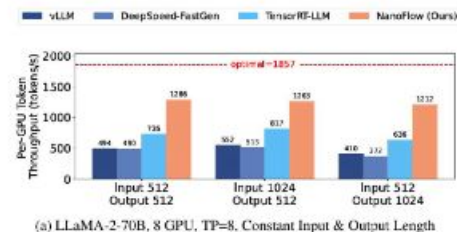


Figure 11: Ablation experiments.



(a) LLaMA-2-70B, 8 GPU, TP=8, Constant Input & Output Length

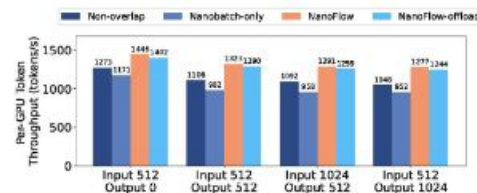


Figure 9: Ablation study results for NanoFlow. Nano-batching and overlapping improves NanoFlow's performance.